

SIMATS ENGINEERING

CO1 – ASSESSMENT TOOL 1

Analytical Problem Solving – Deep Learning (MLA0403)

Course Outcome: CO1 **Marks:** 25 **Duration:** 30 minutes

Name of the Student: K. Vannur Swamy

Reg. No: 192525265

1. Learning Algorithms (Optimization Behavior)

For a convex loss function, gradient descent updates the parameters as $\theta_{t+1} = \theta_t - \eta_t \nabla L(\theta_t)$. A high initial learning rate allows the model to make large movements toward the minimum, so it can reduce the loss quickly during the early stages. Gradually decreasing the learning rate then makes the updates smaller and more stable near the minimum.

This strategy can outperform a constant learning rate when the initial parameters are far from the optimum and a large step size is useful for rapid progress, while smaller steps are required later for accurate convergence. Learning-rate decay is especially useful when a constant rate is either too small, causing slow learning, or too large, causing oscillation around the minimum.

The main risk is that an excessively high initial rate may overshoot the minimum or cause unstable updates. If the learning rate is decreased too quickly, learning may become very slow or stop before reaching a good solution. Thus, the initial rate and decay schedule must be chosen appropriately.

2. Maximum Likelihood Estimation (MLE)

Let $x_1, x_2, \dots, x_n$ be independent observations from $N(\mu, \sigma^2)$, where σ^2 is known. The likelihood is $L(\mu) = \prod [1/(\sqrt{2\pi\sigma^2})] \exp(-(x_i - \mu)^2/(2\sigma^2))$.

Taking the log-likelihood and differentiating with respect to μ gives:

$$\partial \log L / \partial \mu = (1/\sigma^2) \sum (x_i - \mu).$$

Setting this equal to zero gives $n\mu = \sum x_i$. Therefore,

$$\mu_{MLE} = \bar{x} = (1/n) \sum x_i.$$

Outliers: Since the MLE for the Gaussian mean is the sample mean, extreme observations can pull the estimate strongly toward them. Hence Gaussian MLE is not very robust to outliers.

Increasing sample size: As n increases, the sample mean generally becomes more stable and, under the Gaussian model, converges to the true mean μ . Its variance is σ^2/n .

Conclusion: MLE is statistically efficient under the correct Gaussian assumption, but the mean-based estimator is sensitive to outliers.

3. Building a Machine Learning Algorithm (Model Selection)

The dataset has 10,000 features but only 500 samples. This is a high-dimensional, low-sample-size problem. The model can easily learn noise instead of useful patterns, producing severe overfitting. Training can also become computationally expensive and many features may be irrelevant or redundant.

Recommended pipeline:

1. Split the data into training and test sets; use cross-validation within the training set.
2. Standardize the features so that variables are on comparable scales.
3. Perform feature selection using a suitable statistical or model-based method to remove irrelevant features.
4. Apply dimensionality reduction such as PCA when many features contain correlated information.
5. Train a regularized classifier such as logistic regression with L1/L2 regularization.
6. Tune hyperparameters using cross-validation and evaluate the final model only on the held-out test set.

Feature selection reduces noise, PCA compresses the representation, and regularization penalizes unnecessarily complex models. Together these steps reduce variance and improve generalization.

4. Neural Networks (Multilayer Perceptron – MLP)

An MLP with sigmoid hidden neurons computes nonlinear transformations of the input. A hidden layer can be represented approximately as $h_j = \sigma(w_j^T x + b_j)$. The output then combines these hidden representations. Adding more hidden neurons gives the network more adjustable weights and more nonlinear basis functions, allowing it to represent more complex decision boundaries and functions.

However, increased representational power does not guarantee better test performance. With too many neurons relative to the available training data, the network can memorize training examples and noise. This increases variance and causes overfitting: training error may decrease while validation/test error increases.

Therefore, the best network is not necessarily the largest one. Generalization depends on model capacity, data quantity, regularization, validation, and appropriate architecture selection. Techniques such as L2 regularization, dropout, early stopping, and sufficient training data can help control overfitting.

5. Backpropagation, SGD & Curse of Dimensionality

In very high-dimensional data, the number of possible directions in parameter space becomes very large. This affects both optimization and generalization.

Gradient updates: Backpropagation computes gradients for many parameters. A high-dimensional model can have a very large parameter space, increasing computational and memory requirements. Noisy or irrelevant dimensions can also make the gradient less informative.

SGD convergence: SGD uses noisy gradient estimates from individual or small batches. In high dimensions, the optimization landscape may contain many directions with weak or noisy signals, so more iterations may be needed to reach a stable solution.

Generalization: With many features compared with the number of samples, a model has more opportunities to fit noise. This increases the risk of overfitting and poor performance on unseen data.

Techniques:

1. **Dimensionality reduction/feature selection:** PCA or feature selection removes redundant and irrelevant dimensions, reducing computational cost and noise.
2. **Regularization:** L1/L2 penalties discourage overly large or unnecessary weights and reduce overfitting.

Additional methods such as mini-batch training, early stopping, and dropout can also improve optimization stability and generalization.